P.S.V. COLLEGE OF ENGINEERING AND TECHNOLOGY

Krishnagiri, Tamil Nadu

CB3591
ENGINEERING SECURE SOFTWARE SYSTEMS

LABORATORY RECORD

Detail	Entry
Name	
Register Number	
Department	B.E. Computer Science and Engineering (Cyber Security)
Year / Semester	
Academic Year	

Staff Signature: _____ HOD Signature: _____

INDEX

Ex. No	Date	Title of the Experiment	Page	Sign
1		SQL Injection Attack and Prevention		
2		Cross-Site Scripting (XSS) and Prevention		
3		Secure Test Cases (Input Validation)		
4		Buffer Overflow Attack and Prevention		

Note: All programs are run only on the local machine (127.0.0.1 or in memory) for learning. Never test attacks on systems you do not own.

Ex. No: 1
SQL INJECTION ATTACK AND PREVENTION

Date: ____ / ____ / _____

AIM

To demonstrate a SQL injection attack on a login form and to prevent it using parameterized queries.

SOFTWARE REQUIRED

- Windows 10 / 11
- Python 3 (IDLE, VS Code or Command Prompt)
- SQLite (built into Python, no installation needed)

ALGORITHM

1. Create an in-memory SQLite database with a users table and two records.
2. Write a vulnerable login function that joins the user input directly into the SQL string.
3. Write a secure login function that uses ? placeholders (parameterized query).
4. Read the username and password from the user and call both functions.
5. Enter normal and malicious input and compare the two results.

PROGRAM

File name: sqli.py

```
import sqlite3

conn = sqlite3.connect(":memory:")
cur = conn.cursor()
cur.execute("CREATE TABLE users (id INTEGER PRIMARY KEY, username TEXT, password
TEXT)")
cur.executemany("INSERT INTO users (username, password) VALUES (?, ?)",
                [("admin", "admin@123"), ("student", "pass456")])
conn.commit()

def login_vulnerable(user, pwd):
    query = "SELECT * FROM users WHERE username='" + user + "' AND password='" +
pwd + "'"
    print("Query executed:", query)
    return cur.execute(query).fetchone()

def login_secure(user, pwd):
    query = "SELECT * FROM users WHERE username=? AND password=?"
    return cur.execute(query, (user, pwd)).fetchone()

print("=== VULNERABLE LOGIN ===")
u = input("Username: ")
p = input("Password: ")
print("Login SUCCESS" if login_vulnerable(u, p) else "Login FAILED")

print("\n=== SECURE LOGIN ===")
print("Login SUCCESS" if login_secure(u, p) else "Login FAILED")
```

HOW TO RUN

- Open Command Prompt in the folder containing the file.
- Type: python sqli.py
- Enter the test inputs from the test cases table, one run at a time.

TEST CASES

Input (username / password)	Vulnerable login	Secure login
admin / admin@123	SUCCESS	SUCCESS
admin' -- / anything	SUCCESS (attack worked)	FAILED
' OR '1'='1 / ' OR '1'='1	SUCCESS (attack worked)	FAILED

EXPLANATION

In the vulnerable version the input is joined into the query text. With the input admin' -- the quote closes the username string and -- comments out the password check, so the query becomes WHERE username='admin' and the attacker logs in without the password. With ' OR '1'='1 the condition is always true, so every row matches. The secure version passes the input separately from the SQL, so the database treats it only as data and never as code.

OUTPUT

Run 1: normal login

```
=== VULNERABLE LOGIN ===
Username: admin
Password: admin@123
Query executed: SELECT * FROM users WHERE username='admin' AND
password='admin@123'
Login SUCCESS

=== SECURE LOGIN ===
Login SUCCESS
```

Run 2: attack with admin' --

```
=== VULNERABLE LOGIN ===
Username: admin' --
Password: x
Query executed: SELECT * FROM users WHERE username='admin' --' AND
password='x'
Login SUCCESS

=== SECURE LOGIN ===
Login FAILED
```

Run 3: attack with ' OR '1'='1

```
=== VULNERABLE LOGIN ===
Username: ' OR '1'='1
Password: ' OR '1'='1
Query executed: SELECT * FROM users WHERE username='' OR '1'='1' AND
password='' OR '1'='1'
Login SUCCESS

=== SECURE LOGIN ===
Login FAILED
```

RESULT

Thus the SQL injection attack was demonstrated on the vulnerable login and successfully prevented using parameterized queries.

Ex. No: 2
CROSS-SITE SCRIPTING (XSS) AND PREVENTION

Date: ____ / ____ / _____

AIM

To demonstrate a reflected Cross-Site Scripting attack on a web page and to prevent it using output encoding.

SOFTWARE REQUIRED

- Windows 10 / 11
- Python 3 (http.server module, built in)
- Web browser (Google Chrome or Microsoft Edge)

ALGORITHM

1. Start a local web server on 127.0.0.1 port 8000 using Python.
2. Create a /vulnerable page that prints the name parameter directly into the HTML.
3. Create a /secure page that prints the name parameter after html.escape().
4. Open both URLs in the browser with a <script> tag as the name value.
5. Observe that the script runs on the vulnerable page and is shown as plain text on the secure page.

PROGRAM

File name: xss_server.py

```
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import urlparse, parse_qs
import html

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        url = urlparse(self.path)
        name = parse_qs(url.query).get("name", [""])[0]

        if url.path == "/vulnerable":
            body = "<h2>Hello " + name + "</h2>"                # no encoding
        elif url.path == "/secure":
            body = "<h2>Hello " + html.escape(name) + "</h2>"   # encoded
        else:
            body = ('<a href="/vulnerable?name=Guest">Vulnerable page</a> | '
                    '<a href="/secure?name=Guest">Secure page</a>')

        self.send_response(200)
        self.send_header("Content-Type", "text/html; charset=utf-8")
        self.end_headers()
        self.wfile.write(body.encode())

print("Server running at http://127.0.0.1:8000  (Ctrl+C to stop)")
HTTPServer(("127.0.0.1", 8000), Handler).serve_forever()
```

HOW TO RUN

- Type: python xss_server.py and keep the window open.
- Open Chrome or Edge and enter the test URLs.
- Right-click and choose View Page Source to compare the two pages.
- Press Ctrl+C in the Command Prompt to stop the server.
- If Windows Firewall asks for permission, click Allow (only your own PC is used).

TEST CASES

URL	Observation
http://127.0.0.1:8000/vulnerable?name=<script>alert('XSS')</script>	Alert box appears (attack worked)
http://127.0.0.1:8000/secure?name=<script>alert('XSS')</script>	Script shown as text, no alert

EXPLANATION

The vulnerable page inserts user input into the HTML without encoding, so the browser executes the <script> tag. html.escape() converts < into < and > into >, so the browser displays the text instead of running it. Other defences are input validation and the Content-Security-Policy header.

OUTPUT

Command Prompt (server window)

```
Server running at http://127.0.0.1:8000  (Ctrl+C to stop)
127.0.0.1 - - [30/Sep/2026 09:16:16] "GET /vulnerable?
name=%3Cscript%3Ealert('XSS')%3C/script%3E HTTP/1.1" 200 -
127.0.0.1 - - [30/Sep/2026 09:16:16] "GET /secure?
name=%3Cscript%3Ealert('XSS')%3C/script%3E HTTP/1.1" 200 -
```

Browser: vulnerable URL

An alert box appears showing the message XSS. Page source (View Page Source):

Page source of /vulnerable

```
<h2>Hello <script>alert('XSS')</script></h2>
```

Browser: secure URL

No alert box. The page displays the text: Hello <script>alert('XSS')</script>. Page source (View Page Source):

Page source of /secure

```
<h2>Hello &lt;script&gt;alert(&#x27;XSS&#x27;)&lt;/script&gt;</h2>
```

RESULT

Thus the XSS attack was demonstrated and prevented by encoding the output with html.escape().

Ex. No: 3
SECURE TEST CASES (INPUT VALIDATION)

Date: ____ / ____ / _____

AIM

To develop and run security test cases that check whether input validation rejects malicious input.

SOFTWARE REQUIRED

- Windows 10 / 11
- Python 3 (unittest and re modules, built in)

ALGORITHM

1. Write a whitelist validation function for usernames (letters, digits, underscore, 4 to 15 characters).
2. Write a password strength function (length 8 or more, upper, lower, digit and special character).
3. Write unittest test cases with valid input, SQL injection input, XSS input, short input and weak password.
4. Run all the test cases and check that every test passes.

PROGRAM

File name: secure_tests.py

```
import re
import unittest

def is_valid_username(u):
    return bool(re.fullmatch(r"[A-Za-z0-9_]{4,15}", u))

def is_strong_password(p):
    return bool(len(p) >= 8
                and re.search(r"[A-Z]", p)
                and re.search(r"[a-z]", p)
                and re.search(r"\d", p)
                and re.search(r"[^\w\s]", p))

class SecurityTests(unittest.TestCase):
    def test_valid_username(self):
        self.assertTrue(is_valid_username("student_01"))

    def test_sql_injection_username(self):
        self.assertFalse(is_valid_username("admin' --"))

    def test_xss_username(self):
        self.assertFalse(is_valid_username("<script>alert(1)</script>"))

    def test_too_short_username(self):
        self.assertFalse(is_valid_username("ab"))
```

```
    def test_weak_password(self):
        self.assertFalse(is_strong_password("password"))

    def test_strong_password(self):
        self.assertTrue(is_strong_password("Secure@123"))

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

HOW TO RUN

- Type: python secure_tests.py
- Expected output: Ran 6 tests ... OK

TEST CASES

Test	Input	Expected
Valid username	student_01	Accepted
SQL injection	admin' --	Rejected
XSS payload	<script>alert(1)</script>	Rejected
Too short	ab	Rejected
Weak password	password	Rejected
Strong password	Secure@123	Accepted

EXPLANATION

Each test sends valid or malicious input to a validation function and checks the result. Whitelist validation allows only known-good characters, so SQL injection and XSS payloads are blocked before they reach the database or the web page.

OUTPUT

Command: python secure_tests.py

```
test_sql_injection_username
(__main__.SecurityTests.test_sql_injection_username) ... ok
test_strong_password (__main__.SecurityTests.test_strong_password) ... ok
test_too_short_username (__main__.SecurityTests.test_too_short_username) ...
ok
test_valid_username (__main__.SecurityTests.test_valid_username) ... ok
test_weak_password (__main__.SecurityTests.test_weak_password) ... ok
test_xss_username (__main__.SecurityTests.test_xss_username) ... ok

----------------------------------------------------------------------
Ran 6 tests in 0.000s

OK
```

RESULT

Thus the secure test cases were developed and executed, and all 6 tests passed.

Ex. No: 4
BUFFER OVERFLOW ATTACK AND PREVENTION

Date: ____ / ____ / _____

AIM

To show how writing beyond the size of a buffer corrupts adjacent memory, and how bounds checking prevents it.

SOFTWARE REQUIRED

- Windows 10 / 11
- C compiler: Code::Blocks, Dev-C++ or MinGW gcc

ALGORITHM

1. Declare a structure with an 8-byte buffer followed by an integer variable authorized.
2. Read the input and copy it into the buffer using strcpy (no length check).
3. Enter a long input so that the extra characters overwrite the authorized variable.
4. Copy the same input using strncpy with a size limit and a null terminator.
5. Compare the values of authorized in both cases.

PROGRAM

File name: overflow.c

```
#include <stdio.h>
#include <string.h>

struct Data {
    char buffer[8];    /* only 8 bytes */
    int  authorized;   /* sits right after buffer in memory */
};

int main() {
    struct Data d;
    char input[100];
    d.authorized = 0;

    printf("Enter password: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    /* VULNERABLE: no length check */
    strcpy(d.buffer, input);

    printf("buffer     = %s\n", d.buffer);
    printf("authorized = %d\n", d.authorized);

    if (d.authorized != 0)
        printf("ACCESS GRANTED (buffer overflow succeeded!)\n");
    else
        printf("Access denied.\n");

    /* SECURE VERSION */
```

```
    d.authorized = 0;
    strncpy(d.buffer, input, sizeof(d.buffer) - 1);
    d.buffer[sizeof(d.buffer) - 1] = '\0';
    printf("\n[Secure] buffer = %s, authorized = %d\n", d.buffer, d.authorized);
    return 0;
}
```

HOW TO RUN

- Compile: gcc overflow.c -o overflow
- Run: overflow.exe
- In Code::Blocks press F9, in Dev-C++ press F11.
- Enter hello first, then the 10 A's input.

TEST CASES

Input	Vulnerable copy	Secure copy
hello	authorized = 0, access denied	authorized = 0
AAAAAAAAAA (10 A's)	authorized becomes non-zero, ACCESS GRANTED	buffer = AAAAAAA, authorized = 0

EXPLANATION

The buffer holds only 8 bytes but strcpy copies without checking the length. The extra characters spill into the next variable, authorized. The letter A is ASCII 0x41, so the variable becomes non-zero and the check passes. The secure version limits the copy to 7 characters plus a null terminator. Real attacks overwrite the return address on the stack in the same way. Unsafe functions to avoid: strcpy, gets, sprintf, scanf("%s").

OUTPUT

Run 1: normal input

```
Enter password: hello
buffer     = hello
authorized = 0
Access denied.

[Secure] buffer = hello, authorized = 0
```

Run 2: overflow input (10 A's)

```
Enter password: AAAAAAAAAA
buffer     = AAAAAAAAAA
authorized = 16705
ACCESS GRANTED (buffer overflow succeeded!)

[Secure] buffer = AAAAAAA, authorized = 0
```

RESULT

Thus the buffer overflow was demonstrated using strcpy and prevented using strncpy with bounds checking.